

Numerische Methoden — Blueprint

Jede Methode: **Idee/Wofür** (grau) · **Formel** · **Ablauf** (Schritt für Schritt). Voller Stoff der Vorlesungsnotizen „Numerical Methods“ (Kap. 1–7) — nichts darüber hinaus.

1 — Grundlagen & Fehlerarten

Numerik löst, was analytisch nicht (praktikabel) geht — approximativ. Preis: überall Fehler, die man kennen muss.

Transzendente Funktionen (z. B. Sigmoid $\sigma(x) = \frac{1}{1+e^{-x}}$, sin) haben keine geschlossene Lösung $\Rightarrow$ numerisch nötig.

Approximationsfehler

$$\text{Fehler} = |\text{exakter Wert} - \text{Näherungswert}|$$

Fehlerarten — nach Ursache

- **Modellfehler:** Realität vereinfacht.
- **Abbruchfehler:** Verfahren abgeschnitten (Reihe/Gitter).
- **Rundungsfehler:** Maschine speichert ungenau.

Faustfrage: geht's um eine Zahl (Rundung) oder einen Vorgang (Abbruch)?

2 — Diskretisierung & Grid Search

Suchraum in ein Gitter zerlegen, *alle* Kandidaten durchprobieren, den mit dem besten **Zielwert** behalten. Zwei Einsätze: (a) Extremum einer Funktion, (b) ein (Gleichungs-)System lösen.

Stützstellen

$$x_i = a + i h, \quad h = \frac{b-a}{N}, \quad i = 0, \dots, N$$

Zielmaß — was wird minimiert?

- (a) **Extremum:** direkt $f(x_i)$; kleinsten/größten Wert nehmen.
- (b) **System $A\vec{w} = \vec{c}$:** *Residuum $E(\vec{w}) = \sum_j |(Zeile_j \cdot \vec{w}) - c_j|$ (Fehlermaß wählen & begründen!); $\vec{w}$ mit kleinstem E .*

Das Zielmaß braucht nicht die wahre Lösung — E misst nur, wie gut die Gleichungen erfüllt sind. Nicht mit dem Approximationsfehler verwechseln!

Ablauf

Ablauf: 1) Gitter aufstellen (alle x_i bzw. alle $\vec{w}$). 2) Zielmaß überall auswerten. 3) Kandidat mit bestem Zielwert wählen.

Genauigkeit prüfen (separater Schritt!)

Approximationsfehler = |exakt – numerisch| — nur möglich, *wenn* die exakte Lösung bekannt ist (Vergleich, nicht Zielmaß!). *Grobes Gitter $\Rightarrow$ Resultat kann irreführend sein (z. B. Optimum bei $w=0$: dann hängt nichts mehr von w ab — „macht wenig Sinn“). Exakt nur, wenn die Lösung zufällig auf einem Gitterpunkt liegt; feineres h verbessert, garantiert Exaktheit aber nie.*

Komplexität

$$\mathcal{O}(N^d)$$

d Dimensionen — Fluch der Dimensionalität; darum später Newton statt Brute Force.

3 — Gleitkomma (IEEE 754)

Wie der Rechner reelle Zahlen mit endlich vielen Bits speichert — Quelle der Rundungsfehler.

Darstellung

$$x = (-1)^s \cdot m \cdot 2^e$$

s Vorzeichen, m Mantisse, e Exponent. single 1+8+23, double 1+11+52 Bit.

Exponent mit Bias

$$e = E - \text{Bias (lesen)}, \quad E = e + \text{Bias (speichern)}$$

single: Bias 127, $E \in [0, 255]$. Toy-System: Bias frei vorgegeben (z. B. 1).

Zahl $\rightarrow$ IEEE-Form

Ablauf: 1) Zahl binär schreiben. 2) normalisieren $1.m \times 2^e$. 3) $E = e + \text{Bias}$, binär. 4) s , E , m zusammensetzen.

$$10_{10} = 1010_2 = 1.010 \times 2^3$$

$$E = 130 = 10000010_2$$

Maschinengenauigkeit

$$\varepsilon_{\text{mach}} = 2^{-t} \quad (t = \text{Mantissenbits})$$

2^t Werte je Zweierpotenz-Intervall; absolute Lücke $\approx \varepsilon_{\text{mach}}|x|$; relative Präzision $\approx 2^{-t}$. single $\approx 1.19 \cdot 10^{-7}$, double $\approx 2.22 \cdot 10^{-16}$. **Bit-Trade-off:** +Mantisse $\rightarrow$ feinere Auflösung ($\varepsilon_{\text{mach}} \downarrow$); +Exponent $\rightarrow$ größerer Wertebereich.

Eigenes System entwerfen (Klausur 1a)

Ablauf: 1) Bits auf Mantisse/Exp. aufteilen + Trade-off begründen. 2) $\varepsilon_{\text{mach}} = 2^{-t}$. 3) darstellbare Werte $(-1)^s m 2^e$, $e = E - \text{Bias}$. 4) Ausdruck *schrittweise* runden $\Rightarrow$ Auslöschung (§4).

Wertebereich & Festkomma

Überlauf (zu groß) / Unterlauf (zu klein). **Festkomma:** Wert · Skalierung S als Integer speichern, beim Lesen $/S$; kleinste Lücke $1/S$, max. Rundungsfehler $\frac{1}{2S}$ (konstant, $|x|$ -unabhängig). *Gleitkomma = konstante relative Lücke (Dynamik); Festkomma = konstante absolute Lücke (Kontrolle).*

4 — Auslöschung

Subtraktion fast gleicher Zahlen $\Rightarrow$ führende Ziffern weg $\Rightarrow$ großer relativer Fehler.

Vorgehen

Ablauf: 1) beide Zahlen auf gespeicherte Stellenzahl runden. 2) subtrahieren. 3) mit wahrer Differenz vergleichen. *Folge: weniger signifikante Stellen als die Eingaben vortäuschen (Pseudo-Genauigkeit).*

Klausur-Typ $\frac{1}{(1+\varepsilon)-1}$: rundet $1+\varepsilon$ schon auf 1, wird der Nenner

0 statt $\varepsilon \Rightarrow$ Ergebnis explodiert (math. wäre es $1/\varepsilon$). Genau hier zerstört Auslöschung das Resultat.

5 — Fehleranalyse

Wie gut ist ein Verfahren? Vier Eigenschaften messen Empfindlichkeit & Genauigkeit. **Kondition** = Eigenschaft des Problems; **Stabilität/Konsistenz/Konvergenz** = Eigenschaft des Verfahrens.

Symbole: κ Konditionierung, s Stabilität, c Konsistenz (Konvergenz hat keins). f exakt, $\hat{f}$ Verfahren, $\tilde{x} = x \pm \delta$ gestört. *Diskretisierte $\hat{f}$: Punkt erst auf den nächsten Gitterpunkt runden (snappen), dann auswerten.* **Ablauf: je Schrittweite der Reihe nach:** 1) Kondition an ≥ 2 Stellen (flach vs. steil). 2) Stabilität s je h (welche h stabiler?). 3) Konsistenz c je h (welche konsistenter?). 4) Konvergenz = Stab.+Kons.; Fazit: „kleineres h nicht zwingend besser“. *gröberes h meist stabiler, feineres h konsistenter.*

Konditionierung (Problem)

$$\kappa = |f(x) - f(\tilde{x})|$$

Empfindlichkeit des Problems. Klein = gut. $\kappa \approx |f'(x)|\delta$ — je steiler, desto schlechter.

Stabilität (Verfahren)

$$s = \frac{|\hat{f}(\tilde{x}) - \hat{f}(x)|}{|\tilde{x} - x|}$$

Verstärkungsfaktor des Verfahrens. $s \approx 1$: linear.

Konsistenz

$$c = |\hat{f}(x) - f(x)|$$

c = **Konsistenzfehler:** Abstand Verfahren $\leftrightarrow$ Wahrheit bei *exaktem* Input. *Kein fester Wert — man rechnet die Differenz am Punkt aus, das ist c („ $\leq c$ “ = obere Schranke). Kleineres $h \Rightarrow$ kleineres c .*

Konvergenz

$\hat{f}(\tilde{x}) \rightarrow f(x)$ für $h \rightarrow 0$, und

Konvergenz = Stabilität + Konsistenz

Kleineres h verbessert die Konsistenz, kann aber die Stabilität verschlechtern $\Rightarrow$ nicht zwingend bessere Approximation. Grenzwert $\neq f(x)$: systematischer Bias des Verfahrens.

6 — Newton-Raphson

Nullstelle $f(x^*) = 0$, wenn f' bekannt. Tangente legen, ihre Nullstelle nehmen.

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Ablauf: 1) $f(x_n)$ und $f'(x_n)$ *getrennt* ausrechnen. 2) einsetzen $\rightarrow x_{n+1}$. 3) mit x_{n+1} wiederholen bis $|f(x_n)| < \varepsilon$.

Fehlermodus: $f'(x_n) \approx 0$ (Division durch null, falsche Nullstelle, Oszillation). Quadratische Konvergenz: korrekte Stellen verdoppeln sich.

7 — Taylor & Konvergenz

$$f(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(x_n)}{k!} (x - x_n)^k$$

$$= f(x_n) + f'(x_n)(x - x_n) + \frac{f''(x_n)}{2}(x - x_n)^2 + \dots$$

Ablauf: 1) f, f', f'' bei x_n auswerten (gibt Zahlen!). 2) einsetzen — x bleibt Variable, $(x - x_n)$ nicht 0. 3) Ergebnis ist Polynom in x ; erst danach Zahl einsetzen. *Newton = 1. Ordnung (Tangente).*

Quadratische Konvergenz

$$|e_{n+1}| \leq C |e_n|^2, \quad C = \left| \frac{f''(\xi)}{2f'(\xi)} \right|$$

$10^{-1} \rightarrow 10^{-2} \rightarrow 10^{-4} \rightarrow 10^{-8}.$

8 — Sekantenverfahren

Wie Newton, aber f' unbekannt — durch Sekantensteigung ersetzt.

$$x_{n+1} = x_n - f(x_n) \cdot \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

Ablauf: 1) zwei Startwerte x_0, x_1 . 2) $f(x_0), f(x_1)$. 3) Formel $\rightarrow x_2$ (x_n = neuester!). 4) ältesten verwerfen, wiederholen.

Numerische Ableitung (finite Differenz) (wenn f' nicht verfügbar)

$$f'(x_0) \approx \frac{f(x_0 + h) - f(x_0)}{h} \quad (\text{Fehler } \mathcal{O}(h))$$

zentral $\frac{f(x_0+h)-f(x_0-h)}{2h}$, $\mathcal{O}(h^2)$. Genau das ersetzt f' in Newton $\rightarrow$ Sekante; umgeht den Versagensmodus „ f' unbekannt/teuer“.

9 — Newton für Optimierung

Extremum = Nullstelle von f' .

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)}$$

Ablauf: 1) f', f'' bilden. 2) Update iterieren (Newton auf f'). 3) mit f'' -Test klassifizieren.

2.-Ableitungs-Test: $f'' > 0$ Min, $f'' < 0$ Max, $f'' = 0$ unklar.

$|f''|$ -Trick: Nenner $|f''| \Rightarrow$ Schritt immer bergab.

Gradientenabstieg

$$x_{n+1} = x_n - \eta f'(x_n)$$

η Lernrate. SGD: Gradient auf zufälligem Mini-Batch (Monte-Carlo-Schätzung).

10 — Globale Optimierung

Lokale Verfahren bleiben im nächsten Tal hängen — Strategien, um das globale Minimum zu finden.

lokal: $x^* = \arg \min_{x \in N(x_0)} f(x)$

global: $x^* = \arg \min_{x \in \Omega} f(x)$

Konvex ($f'' \geq 0 \forall x$): lokal = global. Testfunktion (kontroll. Multimodalität): Shekel's Foxholes $f(x) = -\sum_i \frac{c_i}{(x-a_i)^2 + r_i}$ — viele lokale Minima, ein globales.

Random Restarts

$$P = 1 - (1 - p)^K, \quad K = \frac{\ln(1 - P)}{\ln(1 - p)}$$

Ablauf: nach K lösen: 1) $(1 - p)^K = 1 - P$. 2) $\ln$ beidseitig. 3) $K = \ln(1 - P) / \ln(1 - p)$. rechts $1 - P$, nicht P !

Basin Hopping

Lokale Optimierung + zufällige Sprünge $\delta \sim D$ vom Minimum aus (kein voller Neustart).

Simulated Annealing

Verschlechterung Δf akzeptiert mit

$$P_{\text{akz}} = \exp(-\Delta f / T), \quad T \downarrow \text{ über Zeit}$$

Stochastik

Rauschen / Mini-Batch glättet die Landschaft, hilft aus lokalen Minima.

Glätten durch Integration (gegen hochfrequente Mini-Minima)

Hochfrequenter Stör-Term (z. B. $+\frac{1}{2} \sin(2\pi x)$) stoppt lokale Optimierer im nächsten Mikro-Tal. Ansatz neben globaler Opt.: **Fenster-Mittel** $\frac{1}{h} \int_{x-h/2}^{x+h/2} f$. $h \approx$ Periode der Störung wählen ($\sin(2\pi x)$: Periode 1 $\Rightarrow h \approx 1$), dann mittelt sie sich heraus.

11 — Numerische Integration

$I = \int_a^b f \, dx$ durch einfache Formen (h Teilbreite, $m = \frac{a+b}{2}$).

Einfache Regeln (1 Intervall, $h=b-a$)

Rechteck (links/rechts): $h f(a)$ bzw. $h f(b)$

Mittelpunkt (bestes Rechteck): $h f(m)$

Trapez: $\frac{h}{2} [f(a) + f(b)]$

Simpson: $\frac{h}{6} [f(a) + 4f(m) + f(b)]$

Steigende Genauigkeit: Rechteck (konstante Höhe) $\rightarrow$ Trapez (Gerade) $\rightarrow$ Simpson (Parabel durch 3 Punkte). Mittelpunkt schlägt Links/Rechts, weil er die Krümmung mittelt.

Zusammengesetzt (n Teilintervalle)

Trapez: $\frac{h}{2} \left[f_0 + 2 \sum_{i=1}^{n-1} f_i + f_n \right]$

Simpson: $\frac{h}{3} \left[f_0 + 4 \sum_{\text{ungerade}} f_i + 2 \sum_{\text{gerade}} f_i + f_n \right]$

Ablauf: 1) h und Stützstellen. 2) alle f_i . 3) mit Gewichten in die Formel.

Fehler (akkum.): Links/Rechts $\mathcal{O}(h)$, Mittelpunkt/Trapez $\mathcal{O}(h^2)$, Simpson $\mathcal{O}(h^4)$. Simpson exakt bis Grad 3.

Lagrange-Interpolation

$$L_j(x) = \prod_{\substack{m=0 \\ m \neq j}}^n \frac{x - x_m}{x_j - x_m}, \quad p(x) = \sum_j y_j L_j(x)$$

Ablauf: 1) jedes L_j (Schalter: 1 am eigenen Knoten, 0 sonst). 2) $\times y_j$, summieren. 3) ausmultiplizieren, Probe $p(x_k) = y_k$. Hoher Polynomgrad oszilliert an den Rändern (Runge-Phänomen) — daher nicht den Grad erhöhen, sondern zusammengesetzte Regeln niedrigen Grades („stop at Simpson“).

12 — Monte-Carlo-Integration

Integral = Volumen $\times$ Mittelwert — durch Zufallspunkte schätzen statt Gitter; dimensionsunabhängig.

$$I = \int_{\Omega} f \, dx = |\Omega| \cdot \mathbb{E}_{X \sim U(\Omega)} [f(X)]$$

$$\hat{I} = \frac{|\Omega|}{N} \sum_{i=1}^N f(X_i)$$

Ablauf: 1) N Zufallspunkte X_i in Ω . 2) $f(X_i)$ alle, summieren. 3) $\times |\Omega|/N$.

$\sigma_f^2 = \frac{1}{N-1} \sum (f(X_i) - \bar{f})^2$, Standardfehler $SE = \frac{|\Omega| \sigma_f}{\sqrt{N}}$. $\mathcal{O}(1/\sqrt{N})$, dimensionsunabhängig; $N \times 4 \Rightarrow$ Fehler halbiert. Mittelpunkregel = Monte-Carlo mit $N=1$ (Sample am Mittelpunkt).

13 — Modelltraining (Synthese)

Jede Standard-Trainingspraxis ist eine der bekannten numerischen Methoden im ML-Gewand. (Notizen Kap. 7; Klausurstil „welche Methode steckt dahinter?“)

Modell & Loss

$$\hat{y}(x) = wx + b, \quad L(w, b) = \frac{1}{n} \sum_{i=1}^n (wx_i + b - y_i)^2$$

Loss bei Initialisierung (Handprobe)

Mit $w=0$, $b=\bar{y}$ sagt das Modell überall $\bar{y}$ voraus:

$$L(0, \bar{y}) = \frac{1}{n} \sum_i (y_i - \bar{y})^2 = \text{Var}(y)$$

Klassifikator (uniform, K Klassen): Cross-Entropy = $\ln K$. Gedruckt $\approx n \cdot \text{Var}$: summiert statt gemittelt; ≈ 0 : falsche Normalisierung.

Overfit-Sanity-Check

Ablauf: 1) kleine Teilmenge + (über)mächtiges Modell. 2) Loss muss $\rightarrow 0$. 3) klappt nicht $\Rightarrow$ Bug in Pipeline (Modell/Loss/Daten), nicht im Optimierer. = Solver an Problem mit bekannter exakter Lösung testen.

Die fünf Korrespondenzen

- Overfit-Test $\leftrightarrow$ Solver an bekannter Lösung (Newton, Kap. 6).
- Initial-Loss $\leftrightarrow$ Randwert-/Handprobe.
- Mini-Batch-SGD $\leftrightarrow$ Monte-Carlo des Gradienten, $SE \propto 1/\sqrt{B}$ (Kap. 12).
- Mehrere Random Seeds $\leftrightarrow$ Random Restarts, nicht-konvex (Kap. 10).
- Quantisierung (Inferenz) $\leftrightarrow$ Präzisionswahl Gleitkomma (Kap. 3); Inferenz trägt weniger Präzision als Training.